

Curs 2 Big Data

Ingestia si Pregatirea Datelor in Big Data

Ingestia si pregatirea datelor sunt pasi esentiali in orice flux de date pentru analiza Big Data. Aceste procese asigura ca datele sunt colectate, curatate, transformate si stocate intr-un mod eficient pentru a fi analizate ulterior. In medii Big Data, precum Apache Spark, Databricks sau solutiile cloud (AWS, Azure, Google Cloud, Oracle Cloud), ingestia si pregatirea datelor sunt fundamentale pentru analiza corecta a acestora.

1. Ingestia Datelor

Ingestia datelor reprezinta procesul de importare a datelor din surse diverse (fisiere, baze de date, API-uri etc.) intr-un sistem unde pot fi procesate, analizate sau stocate pentru utilizare ulterioara.

Pasi principali in ingestia datelor:

- **Identificarea sursei:** Alegeti sursa din care se vor extrage datele (de exemplu, fisiere CSV, loguri, baze de date, senzori IoT, stocare cloud, API-uri).
- **Ingestia pe loturi vs. Ingestia in flux:** Decideti daca datele vor fi ingerate in loturi (ex: la intervale orare sau zilnice) sau in timp real (ex: folosind Kafka, Apache Flink).
- **Extractia datelor:** Extracti datele din sursele alese folosind conectori sau API-uri specifice.
- **Incarcarea datelor:** Incarcati datele intr-un motor de procesare precum Spark, Databricks sau intr-o solutie de stocare cloud (de exemplu, S3, Azure Blob Storage).

Metode comune de ingestie:

- **Procesare pe loturi:** Incarcati volume mari de date la intervale regulate.
- **Procesare in flux:** Incarcati date continuu pe masura ce sunt generate (ex: Kafka, AWS Kinesis).
- **Incarcare directa:** Incarcati date dintr-o sursa externa intr-un sistem tinta, cum ar fi un Data Lake.

2. Curatarea Datelor

Curatarea datelor sau preprocesarea acestora este necesara pentru a asigura ca datele sunt exacte, consistente si utilizabile pentru analiza.

Pasi in curatarea datelor:

- **Eliminarea duplicatelor:** Identificati si eliminati inregistrările duplicate.
- **Gestionarea valorilor lipsa:** Completati valorile lipsa, eliminati randuri sau interpolati datele.
- **Transformarea datelor:** Schimbati tipurile sau formatele de date, agregati datele sau creati coloane derivate.
- **Detectarea valorilor anormale (outliers):** Identificati si gestionati valorile anormale care ar putea afecta analiza.

3. Transformarea Datelor

Transformarea datelor presupune convertirea datelor brute intr-un format care sa fie potrivit pentru analiza.

Pasi in transformarea datelor:

- **Definirea schemei:** Definirea schemei pentru date (ex: structura tabelelor, tipuri de campuri).
- **Normalizarea/Standardizarea:** Normalizarea sau standardizarea datelor pentru a le aduce la o scara comuna.
- **Ingineria caracteristicilor (Feature Engineering):** Crearea de noi caracteristici din datele existente, care ar putea fi mai utile pentru analiza ulterioara (de ex. separarea unui timestamp in data si ora).
- **Imbogatirea datelor:** Adaugarea unor date externe relevante sau a unor coloane calculate.

4. Stocarea Datelor

Dupa ce datele au fost curatate si transformate, trebuie stocate intr-un mod optimizat pentru interogari si procesare eficienta.

Optiuni de stocare:

- **Data Lake:** Folositi un data lake (ex: AWS S3, Azure Data Lake) pentru a stoca date brute sau semi-structurate.
- **Data Warehouse:** Folositi un data warehouse (ex: Amazon Redshift, Google BigQuery, Snowflake) pentru date structurate optimizate pentru interogari analitice.
- **Partiționarea datelor:** Partiționati seturile mari de date in bucati mai mici pentru a imbunatati performanta si gestionabilitatea acestora.

5. Validarea Datelor

Validarea datelor inainte de analiza este esentiala pentru a asigura consistenta si corectitudinea acestora.

Pasi in validarea datelor:

- **Validarea schemei:** Verificati daca datele respecta schema corecta.
- **Verificari de integritate:** Asigurati-va ca datele respecta regulile de business (de exemplu, nu trebuie sa existe valori negative pentru sume).

6. Integrarea Datelor

Integrarea datelor din surse diverse intr-o viziune unificata este cruciala pentru analiza ulterioara.

Metode de integrare a datelor:

- **Joins:** Realizati imbinari intre datele provenite din surse multiple pe baza unor chei comune (de exemplu, join-uri SQL).
- **Sincronizarea datelor:** Pastrati datele actualizate in timp real sau pe baza unor intervale programate.

Exemplu in Databricks (Spark + Delta Lake)

Iata un exemplu de cod in Databricks folosind Apache Spark si Delta Lake pentru ingestia, curatarea si transformarea datelor:

Ingestia datelor dintr-un fisier CSV

```
df = spark.read.csv("/mnt/data/your_data_file.csv", header=True, inferSchema=True)
```

Curatarea datelor: Eliminarea randurilor cu valori lipsa

```
df_clean = df.na.drop()
```

Transformarea datelor: Adaugarea unei noi coloane pe baza unei coloane existente

```
df_transformed = df_clean.withColumn("new_column", df_clean['existing_column'] * 10)
```

Scrierea datelor curatate si transformate in Delta Lake pentru interogari eficiente

```
df_transformed.write.format("delta").mode("overwrite").save("/mnt/delta/transformed_data")
```

Interogarea datelor transformate folosind SQL

```
spark.sql("SELECT * FROM delta.`/mnt/delta/transformed_data` WHERE new_column > 50")
```

Ce inseamna fiecare parte

- **spark.read.csv(...)**
Citeste un fisier CSV intr-un **DataFrame Spark**
- **"/mnt/data/your_data_file.csv"**
Cale din sistemul local (folosita frecvent in Jupyter), **nu este implicita in Databricks**
- **header=True**
Prima linie din CSV este folosita ca nume de coloane
- **inferSchema=True**
Spark detecteaza automat tipurile de date
(int, double, string etc., nu toate coloanele vor fi string)

Practici Recomandate pentru Ingestia si Pregatirea Datelor in Big Data:

- **Parallelism:** Folositi procesarea paralela (ex: Spark) pentru a gestiona volume mari de date eficient.

- **Compresia:** Folositi formate de compresie precum Parquet sau ORC pentru a reduce stocarea si a accelera I/O-ul.
- **Incarcari Incrementale:** Pentru ingestia continua, utilizati incarcari incrementale pentru a procesa doar datele noi sau actualizate.
- **Gestionarea Erorilor:** Implementati o gestionare robusta a erorilor pentru a gestiona probleme precum coruperea datelor sau lipsa fisierelor in timpul ingestiei.

Prin respectarea acestor principii, va puteti asigura ca datele sunt gata pentru analize semnificative, vizualizari si luarea deciziilor in medii Big Data.

2. Curatirea Datelor

Dupa ingestia datelor, acestea trebuie curatate pentru a elimina erorile, valorile lipsa sau anomaliiile.

Pasi pentru curatirea datelor:

- **Eliminarea valorilor lipsa:**
 - Spark poate elimina randurile cu valori lipsa folosind metoda `na.drop()`.
- # Eliminarea valorilor lipsa

```
df_clean = df.na.drop()
```
- **Filtrarea datelor anormale (outliers):**
 - Putem aplica filtrare pentru a elimina datele care nu respecta anumite reguli de afaceri (de exemplu, valori negative pentru campuri financiare).
- # Filtrarea valorilor anormale (ex: campul 'amount' sa nu fie negativ)

```
df_filtered = df_clean.filter(df_clean['amount'] >= 0)
```
- **Transformarea tipurilor de date:**
 - Convertirea unui camp dintr-un tip de data in altul.
- # Schimbarea tipului de date pentru o coloana

```
df_transformed = df_filtered.withColumn("date_column", df_filtered["date_column"].cast("timestamp"))
```

3. Transformarea Datelor

Transformarea datelor presupune crearea unor noi coloane, agregarea datelor sau schimbarea structurii acestora pentru a fi utilizabile in analize.

Pasi pentru transformarea datelor:

- **Adaugarea unei coloane noi:**
 - Putem adauga o coloana bazata pe datele existente.
- # Adaugarea unei noi coloane calculate

```
df_transformed = df_filtered.withColumn("new_column", df_filtered["existing_column"] * 10)
```
- **Agregarea datelor:**

- Spark permite agregarea datelor, de exemplu, calcularea mediei pentru fiecare grup.

```
# Agregarea datelor pe baza unui grup (ex: calcularea mediei pe grupuri)
df_aggregated = df_transformed.groupBy("category").agg({"amount": "avg"})
```

- **Normalizarea sau standardizarea:**

Putem normaliza valorile pentru a le aduce la o scara comuna.

```
# Normalizarea datelor (ex: coloana 'amount' sa fie intre 0 si 1)
from pyspark.ml.feature import MinMaxScaler
from pyspark.ml.linalg import Vectors
from pyspark.sql.functions import col

# Crearea unui vector pentru scalare
scaler = MinMaxScaler(inputCol="amount", outputCol="scaled_amount")
df_scaled = scaler.fit(df_transformed).transform(df_transformed)
```

4. Stocarea Datelor

Dupa curatare si transformare, datele trebuie stocate intr-un mod optimizat pentru a le accesa rapid la analize viitoare. **Databricks** permite salvarea datelor intr-un **Delta Lake**, care optimizeaza stocarea si performanta interogarii.

Pasi pentru stocarea datelor:

1. **Salvarea datelor in Delta Lake:**

- Delta Lake permite salvarea datelor in format optimizat, oferind suport pentru tranzactii ACID si performanta imbunatatita.

```
# Salvarea datelor curatate si transformate in Delta Lake
df_transformed.write.format("delta").mode("overwrite").save("s3a://bucket-name/delta/transformed_data/")
```

- **Scrierea datelor in Redshift:**

- Daca dorim sa incarcam datele procesate intr-un data warehouse precum **Redshift**, putem folosi conectorul JDBC.

```
# Scrierea datelor in Redshift
df_transformed.write.format("jdbc").option("url",
jdbc_url).option("dbtable",
"transformed_table").mode("overwrite").save()
```

5. Integrarea Datelor din Surse Multiple

In multe cazuri, datele provin din surse multiple, si este necesar sa le integram intr-o singura viziune unificata.

Pasi pentru integrarea datelor:

Join intre doua DataFrames:

- Putem imbina datele provenite din surse diferite folosind `join()`.

- # Join intre doua DataFrames
df_joined = df_transformed.join(df_another, on="common_key", how="inner")

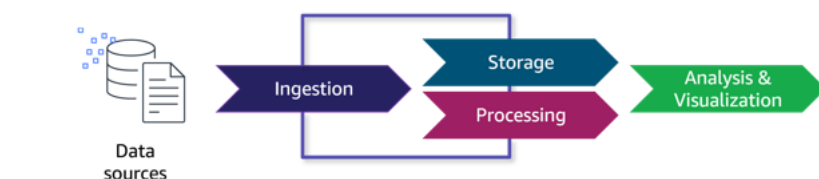
Sincronizarea datelor in timp real:

- In cazul in care dorim sa procesam datele in timp real, putem folosi **streaming** cu Spark.

```
# Citirea datelor dintr-un stream Kafka
df_streaming = spark.readStream.format("kafka").option("kafka.bootstrap.servers",
"localhost:9092").option("subscribe", "topic").load()
```

Prin utilizarea **Databricks**, **AWS** si **Apache Spark**, putem construi un flux de date complet pentru ingestia, curatarea, transformarea si stocarea datelor in medii Big Data. Aceste platforme permit procesarea rapida si scalabila a datelor, indiferent de volumul acestora, si sunt esentiale pentru a face fata cerintelor analizelor moderne de date.

Ingesting data involves storing and processing data



© 2022, Amazon Web Services, Inc. or its affiliates. All rights reserved.

6

Fig. 1 Ingesting data

Figura anterioara ilustreaza un flux de lucru tipic pentru procesarea datelor intr-un mediu de Big Data. Aceasta descrie pasii principali dintr-un pipeline de date, incepand cu sursele de date si ajungand la analiza si vizualizarea acestora. Iata ce reprezinta fiecare componenta:

1. **Data Sources (Surse de date):**
 - Reprezinta sursele de unde provin datele. Acestea pot fi baze de date, fisiere, fluxuri de date in timp real, API-uri sau alte surse externe.
2. **Ingestion (Ingestia):**
 - Este procesul prin care datele sunt colectate si aduse intr-un sistem de procesare. Ingestia poate fi facuta in **loturi** sau in **timp real**. Instrumentele populare pentru ingestie includ Apache Kafka, AWS Kinesis, sau Databricks.
3. **Storage (Stocarea):**
 - Dupa ingestie, datele sunt stocate intr-un loc centralizat pentru a fi procesate ulterior. Acestea pot fi stocate intr-un **Data Lake** sau intr-un **Data Warehouse**. Solutii precum **AWS S3**, **Azure Blob Storage**, sau **Google Cloud Storage** sunt utilizate frecvent.
4. **Processing (Procesarea):**

- Datele stocate sunt prelucrate pentru a le curata, transforma si agrega, pentru a le face utile in analiza ulterioara. Acest pas include operatiuni precum curatarea datelor, transformarea lor intr-un format utilizabil si efectuarea de calcule. Apache Spark si Databricks sunt adesea folosite pentru acest pas.

5. Analysis & Visualization (Analiza si Vizualizarea):

- Dupa procesare, datele sunt analizate si vizualizate pentru a extrage informatii valoroase. Aceasta poate include utilizarea de instrumente de BI (Business Intelligence) cum ar fi Tableau, Power BI, sau librarii Python ca **Matplotlib**, **Seaborn** sau **Plotly** pentru vizualizarea rezultatelor analizei.

Aceasta diagrama reprezinta fluxul clasic de procesare a datelor de la colectare (ingestie), trecand prin stocare si procesare, pana la analiza finala si vizualizare, care este esentiala pentru luarea deciziilor bazate pe date.

In practica, procesul de ingestie a datelor presupune preluarea datelor dintr-o sursa externa si introducerea acestora in pipeline-ul tau, unde vor putea fi utilizate. Procesul de ingestie este legat de sarcinile de stocare si procesare a datelor.

In mod specific, procesul de ingestie presupune extragerea datelor dintr-o sursa care nu este deja disponibila in pipeline, incarcarea datelor intr-un tip de stocare temporara in pipeline si procesarea acestora prin transformarea lor inainte sau dupa ce au fost incarcate intr-un stocaj permanent.

Tipul si amploarea transformărilor si locul in care acestea se realizeaza pe parcursul pipeline-ului de date depind in mare masura de necesitatile business-ului si de natura datelor care sunt ingerate.

Procesul de ingestie a datelor intr-un pipeline implica preluarea datelor din surse externe, stocarea temporara si procesarea acestora inainte de a le incarca intr-un stocaj permanent. Transformarile datelor depind de nevoile de business si de caracteristicile datelor.

ETL and ELT

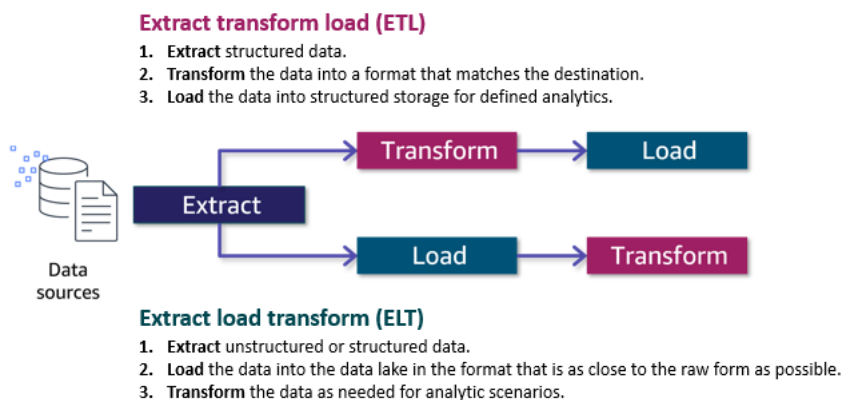


Fig. 2 ETL and ELT

Aceast imagine (Fig. 2 ETL si ELT) prezinta doua metode de ingestie a datelor. Procesul implica extragerea datelor dintr-o sursa, urmat de incarcare in stocare si transformarea lor pentru a fi utilizate in scopuri analitice.

Metoda traditionala de ingestie si pregatire a datelor pentru analize a fost **ETL** (Extract, Transform, Load). In acest proces, datele sunt extrase din sursa lor, transformate intr-un format structurat si apoi incarcate intr-un depozit de date (ex: un data warehouse). Analistii sau oamenii de stiinta a datelor interogheaza depozitul de date si pot efectua transformari suplimentare pentru analize.

Pe masura ce depozitele de date s-au extins pentru a gestiona nu doar date structurate, ci si date nestructurate sau semi-structurate (in **data lakes**), instrumentele si metodele de ingestie au evoluat. Tehnicile ETL raman valoroase pentru datele structurate, dar exista o tendinta de a colecta tot mai mult date nestructurate si semi-structurate.

Astfel, a aparut o noua metoda de ingestie a datelor, **ELT** (Extract, Load, Transform), in care datele sunt extrase din sursa, transformate minim pentru a fi stocate intr-un **data lake** intr-o stare relativ bruta, iar transformarile suplimentare sunt facute ulterior, pe masura ce utilizatorii acceseaza datele si dezvoltat solutii analitice.

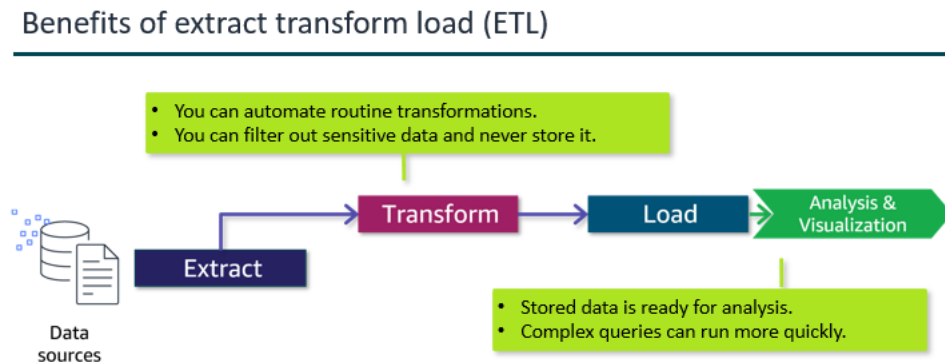


Fig. 3 Benefits of ETL

Pe langa faptul ca ETL functioneaza bine pentru date structurate destinate unui data warehouse, iar ELT este mai potrivit pentru date nestructurate care ajung intr-un data lake, exista si alte motive pentru a alege una dintre cele doua metode. Imaginea anterioara subliniaza cateva aspecte de luat in considerare.

ETL stocheaza datele care sunt deja pregatite pentru analiza, ceea ce poate economisi timp pentru un analist. Daca analistii efectueaza frecvent aceleasi transformari pe datele pe care le analizeaza, ar putea fi mai eficient sa le integreze direct in procesul de ingestie. Astfel, folosind ETL, faci schimbul dintre un timp mai lung de procesare a transformarilor inainte ca datele sa fie disponibile si un acces mai rapid atunci cand esti pregatit sa le interoghezi.

Un alt avantaj al realizarii transformarilor dinainte este ca poti filtra informatiile personale identificabile (PII) sau alte date sensibile care ar putea crea riscuri de conformitate. Daca nu stochezi niciodata datele sensibile, reduci riscul fara a fi nevoie de masuri suplimentare de securitate.

Benefits of extract load transform (ELT)

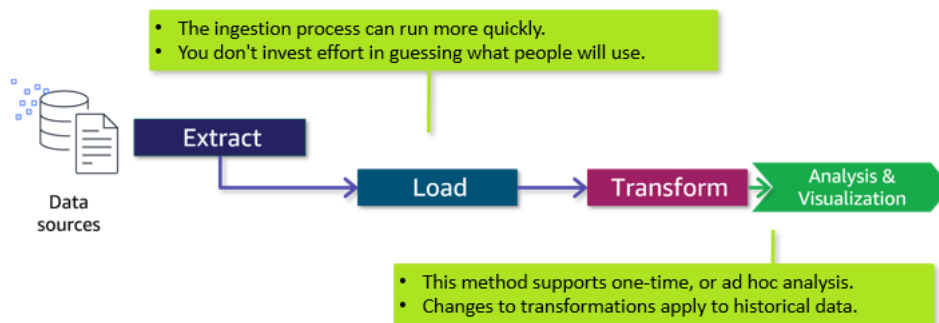


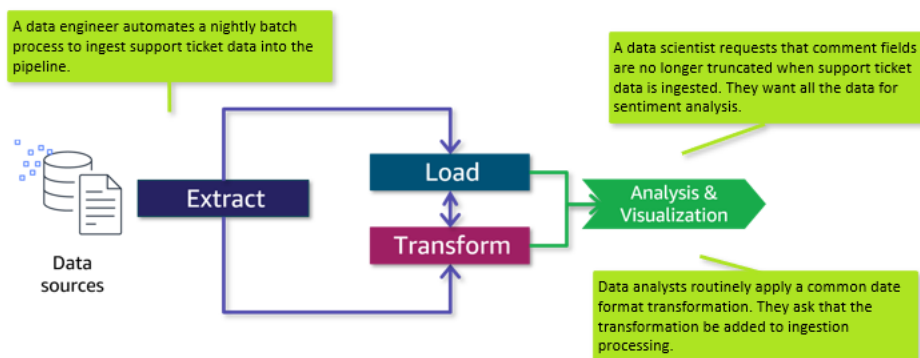
Fig 4. Benefits of ELT

Pe de alta parte, **ELT** ofera mai multa flexibilitate in crearea de interogari noi, deoarece analistii au acces la cantitati mai mari de date brute.

Cu **ELT**, procesul de a aduce datele brute in pipeline poate fi mai rapid, deoarece nu este necesar sa transformi datele atat de mult inainte de a le face disponibile in **data lake**. Astfel, cu **ELT**, compromisul este ca obtii datele brute mai repede, dar vei petrece mai mult timp transformandu-le atunci cand vrei sa lucrezi cu ele.

Un alt avantaj al stocarii datelor brute si transformarea acestora doar ulterior este ca modificarile la transformarile existente (de exemplu, adaugarea unor campuri suplimentare sau formatarea diferita a datelor) se vor aplica la toate datele istorice deja incarcate atunci cand actualizezi transformarea. Daca transformarile au loc inainte de incarcare, modificarile vor afecta doar datele noi, iar aplicarea acestora la datele istorice poate fi imposibila sau dificila.

The ingestion process evolves as the pipeline is used



© 2022, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Fig. 5 The ingestion process evolution

Procesul de ingestie a datelor nu are o forma standard valabila in orice situatie, ci este influentat direct de obiectivele de business si de alegerile facute pentru fiecare scenariu in parte. Pe masura ce experienta echipei creste si utilizatorii interactioneaza mai mult cu pipeline-ul, modul in care acesta este construit poate fi ajustat si imbunatatit.

In arhitecturile moderne, granitele dintre ETL si ELT devin tot mai greu de separat. Diferitele roluri implicate in lucrul cu datele contribuie in etape diferite la ingestie si transformare, folosind instrumente si niveluri de acces distincte. De exemplu, data engineerul poate realiza

transformari de baza si curatarea datelor chiar inainte de incarcarea lor in data lake. Alte transformari pot fi aplicate ulterior, atunci cand datele sunt folosite pentru analize concrete. Data analystul adapteaza datele pentru rapoarte specifice, in timp ce data scientistul continua explorarea si prelucrarea lor pentru a descoperi tipare sau relatii relevante.

Odata cu evolutia modului de utilizare a datelor, devine necesara reevaluarea locului in care sunt efectuate transformarile. Daca anumite prelucrari sunt realizate frecvent de mai multi utilizatori, includerea lor mai devreme in pipeline poate aduce beneficii de performanta si cost. In alte cazuri, reducerea transformarilor initiale poate crea mai multa flexibilitate. De exemplu, o limitare aplicata unui camp pentru a economisi resurse poate elimina informatii utile, iar acest lucru poate fi observat ulterior de un data scientist, care va solicita pastrarea datelor complete. Rolul data engineerului este sa urmareasca constant comportamentul pipeline-ului, felul in care datele sunt consumate si sa ajusteze procesele astfel incat valoarea obtinuta din date sa fie maximizata.

Ingestia datelor inseamna aducerea acestora in pipeline si aplicarea unor transformari asupra lor. Atunci cand se foloseste un flux ETL, acesta este potrivit in special pentru date structurate, care urmeaza sa fie incarcate intr-un data warehouse sau intr-un alt tip de stocare structurata. In schimb, un flux ELT este mai eficient pentru date nestructurate, care sunt pastrate intr-un data lake si pot fi reutilizate in multiple scenarii de analiza. In cadrul unui pipeline, modul de procesare ETL sau ELT trebuie adaptat si ajustat in timp pentru a obtine valoarea maxima din date.

Data wrangling (datele se fac utilizabile)

Data wrangling (numit si **data munging**) este procesul prin care **datele brute** sunt curatate, transformate si structurate astfel incat sa poata fi **analizate sau folosite in modele**.

Acesta reprezinta procesul de a prelucra cantitati mari de date brute, fie ele structurate sau nestructurate, provenite din surse diferite cu scheme variate, pentru a obtine un set de date coerent si valoros, util pentru procesele ulterioare sau pentru utilizatori.

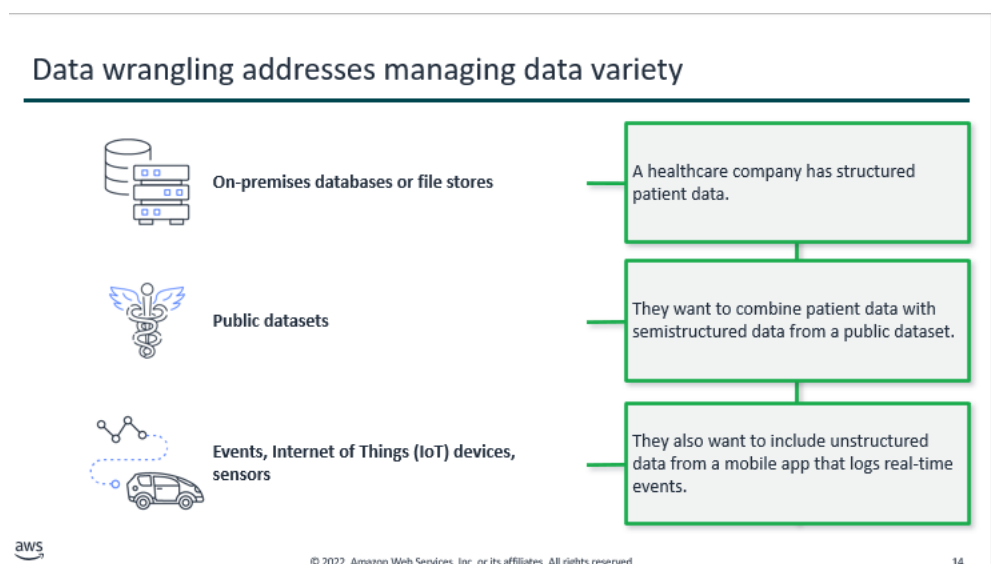


Fig. 6 Data wrangling addresses managing data variety

Fig. 6 prezinta cum o organizatie poate combina diferite tipuri de date pentru a crea o experienta personalizata pentru clienti. De exemplu, poate interoga date dintr-o baza on-premises pentru a identifica pacientii care au intarziat la o vizita.

Ideea esentiala: combinarea diferitelor surse de date permite organizatiilor sa obtina informatii relevante si personalizate pentru utilizatori.

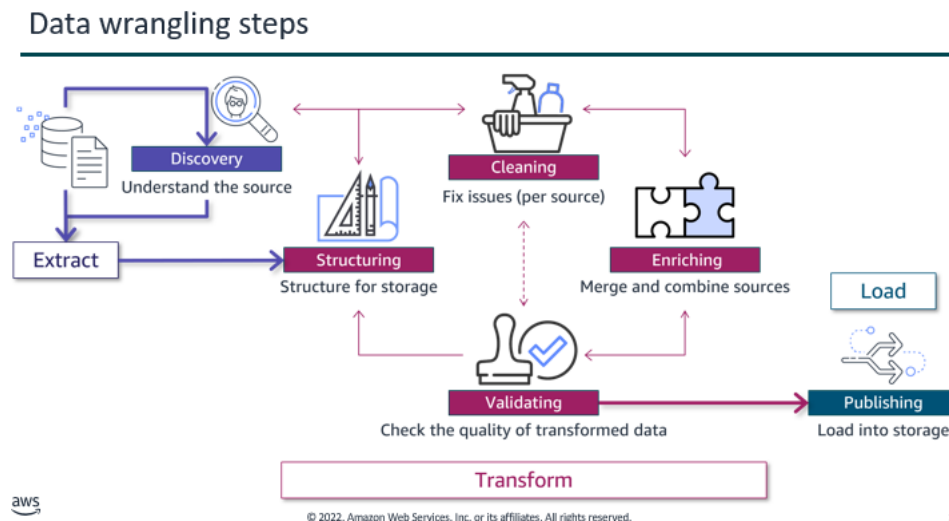


Fig. 7 Data wrangling steps

Imaginea arata ca **data wrangling** face parte din procesul de ingestie a datelor, desi pasii nu sunt intotdeauna strict separati si pot fi repetati sau varia in complexitate. Procesul poate fi simplu, cum ar fi modificarea manuala a unui fisier Excel si incarcarea lui, sau complex, implicand transformari pentru volume mari de date in aplicatii de tip ML.

Data wrangling se suprapune cu ETL, dar exista diferente:

- **ETL:** focus pe data engineer, transformari automate, date deja curate.
- **Data wrangling:** mai apropiat de ELT, permite data scientistilor si utilizatorilor de business sa prelucreze date brute direct in pipeline.
- Trendul in cloud este de a abstractiza infrastructura si codul, oferind mai multa autonomie utilizatorilor pentru a extrage si transforma date.
- Data engineerii trebuie sa construiasca pipeline-uri complete, dar este important sa profite si de tool-uri care ofera flexibilitate analistilor si data scientistilor.

Data wrangling este procesul in mai multi pasi prin care date brute, structurate sau nestructurate, provenite din surse multiple, sunt transformate pentru a fi utilizate in analize sau modele ML. Este esential pentru data scientistii care construiesc modele. Pasii pot fi iterativi, suprapusi sau chiar omisi in unele procese si includ: **descoperire, structurare, curatare, imbogatire, validare si publicare.**

Data discovery (descoperirea datelor)

Data discovery este un pas iterativ al data wrangling, in care se identifica sursele de date utile si se evalueaza valoarea lor pentru business. Data engineer-ul analizeaza relatiile, formatele si intervalele de date, stabileste modul de organizare si acces, si decide ce instrumente sunt necesare pentru stocare, securizare si transformare.

Activitati cheie:

- Evaluarea utilitatii sursei pentru obiectivele business
- Identificarea relatiilor dintre date si intre surse
- Determinarea formatelor de date (CSV, Parquet, ORC)

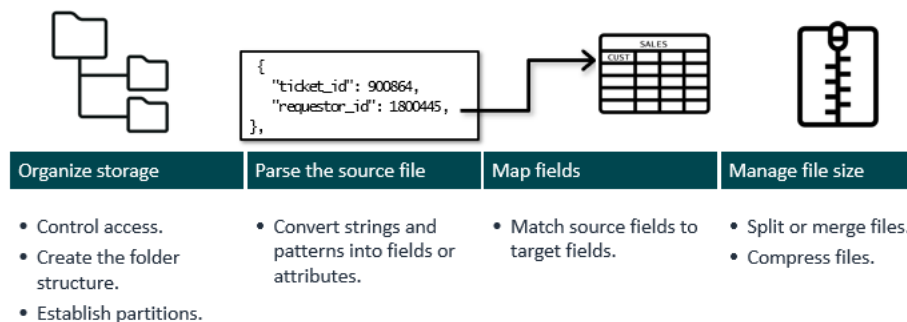
- Stabilirea intervalului de date necesar si modalitatea de extragere
- Organizarea datelor pentru acces eficient si sigur, optimizarea structurii (partitionare, folder structure, zone raw/landing/trusted)
- Alegerea instrumentelor si resurselor necesare pentru stocare, securizare, transformare si interogare

Pe baza acestei analize, se decide daca sursa merita utilizata sau daca e nevoie sa se caute alte surse sau sa se optimizeze volumul si complexitatea datelor.

Data structuring (structurarea datelor)

Data structuring presupune transformarea datelor brute din fisierele sursa intr-un format care permite combinarea si stocarea lor impreuna cu alte date. Aceasta include organizarea stocarii si a accesului, parsarea fisiereilor sursa si maparea campurilor catre structura tinta. De asemenea, data structuring implica aplicarea de strategii pentru optimizarea dimensiunii fisiereilor, cum ar fi impartirea sau comprimarea acestora.

Decisions and tasks in data structuring

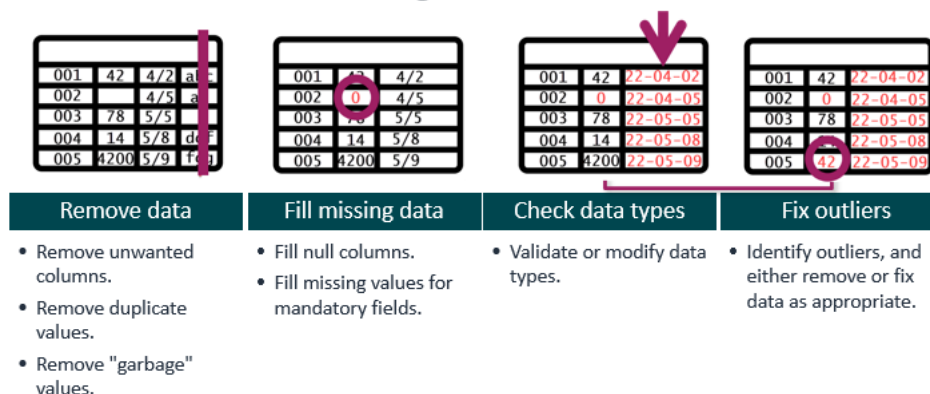


© 2022, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Fig. 8 Decisions and tasks in data structuring

Data cleaning

Decisions and tasks in cleaning



© 2022, Amazon Web Services, Inc. or its affiliates. All rights reserved.

3

Fig. 9 Tasks in cleaning

Data cleaning reprezinta procesul de pregatire a datelor sursa pentru a fi utilizate intr-un pipeline. Curatarea se face pentru fiecare sursa, adaptandu-se la caracteristicile ei. Aceasta include eliminarea datelor inutile, duplicate sau invalide, precum si corectarea valorilor lipsa si a tipurilor de date.

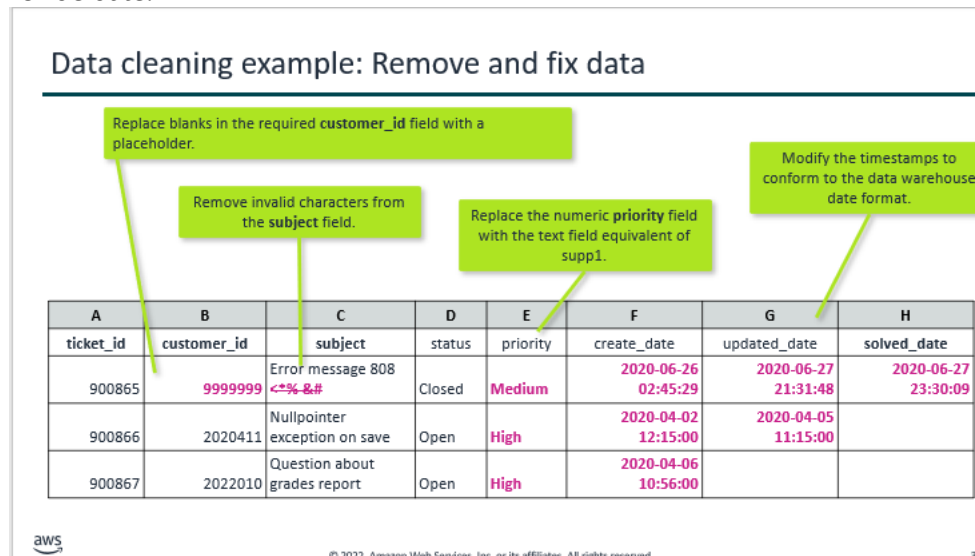


Fig. 10 Data cleaning example

In exemplul dat, pasul de **data cleaning** include inlocuirea valorii lipsa pentru `customer_id` cu un placeholder care respecta numarul corect de cifre, astfel incat setul de date sa nu esueze la incarcare in data warehouse. In functie de caz, data engineer-ul poate transmite fisierul unui data analyst pentru corectarea directa a `customer_id`.

Un alt exemplu de curatare este eliminarea datelor corupte din coloana C, care au fost lipite in campul subject dintr-un mesaj de eroare. Data engineer-ul recunoaste ca anumiți caractere speciale pot crea probleme la incarcare sau la analiza datelor.

Ultimul pas de curatare in acest exemplu este validarea si ajustarea tipurilor de date. Data engineer-ul modifica valorile numerice de prioritate (1, 2, 3) pentru a corespunde valorilor text din celalalt sistem de ticketing: 1 → high, 2 → medium, 3 → low. De asemenea, el ajusteaza campurile de data pentru a respecta formatul de timestamp folosit in data warehouse: YYYY-MM-DD HH:MI:SS.